

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

BizIQ – System Analysis Documentation (SAD) | UMHackathon 2026

1. Introduction	2
1.1 Purpose	2
1.2 Background	2
1.3 Target Stakeholders	3
2. System Architecture & Design	3
2.1 High Level Architecture	3
2.2 LLM as Service Layer	4
2.3 Technological Stack	5
3. Key Data Flows	6
3.1 Data Flow Diagram (DFD)	6
3.2 Normalized Database Schema	7
4. Functional Requirements & Scope	8
4.1 Minimum Viable Product (MVP)	8
4.2 Non-Functional Requirements (NFRs)	9
4.3 Out of Scope / Future Enhancements	10
5. Monitor, Evaluation, Assumptions & Dependencies	10
5.1 Technical Evaluation (Grayscale Rollout & A/B Testing)	10
5.2 Emergency Rollback & Golden Release	11
5.3 Monitoring	11
5.4 Assumptions	12
5.5 External Dependencies	12
6. Project Management & Team Contributions	13
6.1 Project Timeline	13
6.2 Team Members Role	14
6.3 Recommendations	14

1. Introduction:

This section introduces BizIQ, an AI-Powered SME Decision Intelligence Platform developed by Team Nexus (Antigravity AI) for UMHackathon 2026. BizIQ is the strategic solution to the problem statement highlighted in the Product Requirement Document (PRD): small and medium-sized enterprise (SME) business owners lack accessible tools to transform raw business data into actionable decisions. BizIQ bridges this gap by combining a KPI dashboard, AI chat assistant, financial simulation engine, and decision recommendation engine into a single unified web-based platform.

1.1. Purpose:

The System Analysis Document highlights the technical scope and design decisions behind the BizIQ development. The key elements of the system covered are as follows:

- 1.2. Architecture:** *BizIQ is a client-server web application. The frontend is a React 19 SPA (Single Page Application) bundled with Vite and served by an Express.js backend on Node.js. The backend also acts as an AI proxy layer, forwarding structured prompts to the Z AI GLM via the OpenAI-compatible SDK.*
- 1.3. Data Flows:** *Business data (CSV/Excel files) uploaded by the user is parsed server-side using Multer and XLSX/PapaParse libraries, aggregated into KPI metrics (revenue, margin, top/bottom products), and the AI responses are streamed back to the frontend via Server-Sent Events (SSE) in real time.*
- 1.4. Model Process:** *The system follows a multi-step prompting pipeline: (1) raw data is parsed into structured KPI metrics, (2) the AI generates insights and recommendations, and (3) results are rendered on the dashboard and chat interface for the user.*
- 1.5. Role of Reference:** *This document serves as the primary technical reference for all developers, evaluators, and stakeholders. It covers architecture, data flows, technology stack, functional requirements, monitoring strategy, and team contributions for the BizIQ project.*

2. Background:

SMEs across Malaysia rely heavily on manual processes such as handwritten ledgers and raw Excel spreadsheets, with limited tools to interpret business data or incorporate external economic factors. Traditional dashboards provide descriptive analytics but fail to guide business owners on what actions to take.

- 2.1. Previous Version:** *No prior version exists. BizIQ v1.0 is developed from scratch for UMHackathon 2026 as a new AI-driven decision intelligence platform targeting SME retail and small-scale operations.*
- 2.2. Changes in Major Architectural Components:** *As a new build, all architectural components are introduced for the first time in this version: data ingestion via CSV/Excel upload, KPI dashboard with Recharts, AI chat powered by Z AI GLM, financial simulation engine, and decision recommendation engine.*

3. Target Stakeholders

Stakeholders	Roles	Expectations
SME Business Owner	Uploads CSV/Excel business data, interacts with AI assistant, runs financial simulations, and reviews KPI dashboards to make informed decisions.	Simple data upload, plain-language AI insights, reliable scenario predictions, and actionable recommendations without requiring technical expertise.
E-commerce Seller	Reviews product-level performance metrics, evaluates pricing strategies, and monitors gross margin trends across their product catalogue.	Clear product performance breakdowns, pricing suggestions, and margin impact forecasts.
Entrepreneur / Startup Founder	Uses the financial simulation engine to model business scenarios and evaluate strategic options before committing resources.	Accurate What If projections for pricing, COGS, and marketing spend adjustments.
Development Team (Nexus)	Builds and maintains the full-stack platform, integrates the Z AI GLM API, and ensures robust data parsing and fallback mechanisms.	Clear API contracts, modular codebase, structured prompt pipeline, and documented architecture.
UMHackathon Judges / Evaluators	Assess technical feasibility, AI integration quality, system design, and business impact of the BizIQ platform.	Transparent architecture documentation, a working demo, and clearly articulated design decisions aligned with the SAD template.

4. System Architecture & Design

4.1. High Level Architecture

4.2. Overview:

Type	Details
System	Web Application (Browser-based SPA + Node.js Server)
Architecture	Client-Server with AI Proxy Layer (Monolithic Node.js backend with Express)
Deployment	Local / Self-hosted (localhost:3000); production-ready for VPS or cloud hosting
AI Integration	Z AI GLM via OpenAI-compatible SDK for context-aware reasoning and structured output generation

BizIQ is structured as a client-server application with three distinct layers. The React 19 frontend handles file uploads, KPI dashboard visualizations (Recharts), AI chat interface (SSE streaming), and the financial simulation UI. The Express.js backend acts as both the API server and AI proxy: it receives uploaded files, parses them with XLSX and PapaParse, aggregates the data into KPI summaries, constructs structured prompts, and forwards them to the Z AI GLM API. AI responses are streamed back to the frontend in real time via Server-Sent Events (SSE).

4.3. LLM as Service Layer

The Z AI GLM is integrated as a dedicated service layer within the Express backend (server.ts). It is not a generic AI Module but a purpose-built reasoning engine invoked at specific, well-defined points in the data processing pipeline.

4.4. Dependency Diagram

Component	Description
React Frontend (Vite SPA)	Sends HTTP POST /api/upload with CSV/Excel file. Connects to GET /api/chat/stream via SSE for streaming AI responses. Renders dashboard charts from GET /api/analyze JSON response.
Express Backend (server.ts)	Receives file via Multer middleware. Invokes XLSX or PapaParse to parse and aggregate data. Builds structured prompt (KPI JSON + user query). Calls Z AI GLM via OpenAI SDK with stream: true. Pipes GLM token chunks back to client via SSE.
Prompt Construction	Raw CSV/Excel -> Aggregation (Top-N products, revenue totals, margin summaries) -> JSON KPI object -> Injected into system prompt. User chat query injected as user message.
Context Window	System prompt: Business KPI summary in JSON (~500-800 tokens). User message: Natural language query or simulation parameters. Total per request: ~1,000-

	2,500 tokens. Token limit enforced via Top-N filtering and data truncation before API call.
Response Parsing	GLM API returns streamed JSON delta chunks. Backend pipes chunks to SSE stream (/api/chat/stream). Frontend SSE listener appends tokens to chat bubble in real time for immediate perceived responsiveness.
Token Limit Enforcement	Applied before API call: datasets exceeding 1,000 rows are summarized into aggregated metrics. Only top 5 and bottom 5 products by revenue are passed. Raw data rows are never sent to the GLM model.

4.5. Sequence Diagram - AI Chat Flow (Place Query Flow)

Step	Actor	Action
1	User (Browser)	Uploads CSV/Excel file via drag-and-drop on the Dashboard page.
2	Frontend -> Backend	HTTP POST /api/upload with multipart file payload sent to Express server.
3	Backend (Multer)	File saved to /uploads/ directory (temporary session storage).
4	Backend (Parser)	XLSX or PapaParse processes file into JavaScript row array. Aggregation engine computes: total revenue, gross margin, YoY growth, top/bottom 5 products.
5	Backend -> Frontend	Returns structured KPI JSON. Frontend renders Recharts dashboard (line, bar, area charts from normalized metric objects).
6	User (Browser)	Types a business query in the AI Chat interface (e.g., Which products should I discontinue?).
7	Frontend -> Backend	POST /api/chat with { query, sessionKPI } payload.
8	Backend (Prompt Builder)	Constructs system prompt: injects KPI summary JSON + user query as user message. Applies Top-N filter if token count exceeds limit.
9	Backend -> Z AI GLM	OpenAI SDK call to GLM endpoint with stream: true parameter enabled.
10	Z AI GLM -> Backend	Streamed token delta chunks returned via SSE events from the GLM API.
11	Backend -> Frontend	Token chunks piped to GET /api/chat/stream SSE endpoint for the browser.
12	Frontend (SSE Listener)	Appends each token to chat bubble in real time. User sees the AI response stream appear progressively.

4.6. Technological Stack

Layer	Technology	Justification
Frontend	React 19 + Vite	Fast HMR, lightweight SPA bundling, modern JSX. Powers all dashboard, chat, and simulation UI pages.

Styling	Tailwind CSS	Utility-first CSS for rapid responsive design. Enables the premium Modern Dark aesthetic across all UI components.
Animations	Framer Motion	Smooth page transitions and micro-interactions for professional UX feel throughout the platform.
Charts	Recharts	React-native chart library for KPI visualizations (line, bar, area charts). Renders from normalized metric objects, not raw AI output.
Icons	Lucide React	Consistent, lightweight SVG icon set for all dashboard and navigation UI elements.
Backend	Node.js + Express	Lightweight RESTful API server handling file uploads, data parsing, prompt construction, and GLM API proxying.
File Upload	Multer	Express middleware for multipart/form-data. Stores uploaded files temporarily in /uploads/ for session processing.
Data Processing	XLSX + PapaParse	XLSX parses Excel files (.xlsx); PapaParse handles CSV files. Both convert raw uploads into JavaScript row arrays for aggregation.
AI Integration	OpenAI SDK v6.34.0	OpenAI-compatible SDK configured to call Z AI GLM endpoint. Supports streaming responses via SSE for real-time chat output.
Streaming	Server-Sent Events (SSE)	Lightweight one-directional streaming from Express server to browser. Pushes AI token chunks to the chat UI in real time.
Runtime (TypeScript)	TSX	Enables seamless TypeScript execution for server.ts without a separate compilation step during development.
Dev Tooling	npm workspaces	Root package.json orchestrates dev/build/install scripts across the monorepo structure (root + biziq sub-directory).

5. Key Data Flows

BizIQ processes two types of inputs: structured business data (CSV/Excel files) and unstructured user inputs (natural language queries and simulation parameters). All data flows through a strictly controlled pipeline to ensure token efficiency and consistent output quality.

5.1. Data Flow Diagram (DFD):

BizIQ does not use a persistent database. All parsed data and KPI summaries are stored in-memory per session. This is consistent with the PRD constraint of static uploaded datasets with no real-time integrations. Data is cleared when the session ends.

External Entity	Process	Data Store	Output
Mock DataSet	1.0 Upload & Parse	Temp /uploads/ folder (Multer)	KPI JSON: revenue, margin, growth, top/bottom 5 products
Mock DataSet	2.0 AI Query Processing	Session KPI Context (in-memory)	Streaming AI response: insights, recommendations, explanations
Mock DataSet	3.0 Simulation Engine	Session KPI Context (in-memory)	Projected profit, revenue, and margin impact values from What If inputs
Mock DataSet	4.0 Decision Engine	Session KPI Context (in-memory)	Anomaly flags and actionable recommendations: pricing, inventory, cost reduction

5.2. Normalized Database Schema:

Upon upload, raw CSV/Excel rows are normalized into the following internal KPI schema before being passed to any downstream process. Missing values are handled via rule-based imputation or row exclusion validated before any AI API call is made.

Field	Type	Description
totalRevenue	Number	Sum of all revenue rows across the dataset.
totalCOGS	Number	Sum of all cost-of-goods-sold entries in the dataset.
grossMargin	Number (%)	Calculated as: $(\text{totalRevenue} - \text{totalCOGS}) / \text{totalRevenue} \times 100$.
yoyGrowth	Number (%)	Year-on-year revenue growth rate, computed if multi-period data is present.

topProducts[]	Array<{name, revenue, margin}>	Top 5 products by revenue passed to AI for insight generation.
bottomProducts[]	Array<{name, revenue, margin}>	Bottom 5 products by revenue flagged as underperformers for recommendations.
monthlyTrend[]	Array<{month, revenue}>	Monthly revenue time-series for Recharts line/area dashboard charts.

6. Functional Requirements & Scope

This section highlights the core boundaries of the BizIQ project, enabling the team to focus on a high-impact MVP to showcase the technical feasibility of the platform.

6.1. Minimum Viable Product:

#	Features	Description
1	Data Ingestion Module	Users drag and drop CSV or Excel (.xlsx) files. The backend parses the file using XLSX or PapaParse, validates schema fields (revenue, cost, product name), aggregates metrics into a KPI JSON object, and returns it to the frontend for dashboard rendering.
2	Business KPI Dashboard	Displays total revenue, gross margin, and YoY growth as headline KPIs. Renders monthly revenue trend charts, product performance bar charts, and margin breakdowns using Recharts. Updates in real time after each successful file upload.
3	AI Business Assistant	A streaming chat interface powered by Z AI GLM. The user types a natural language query; the system injects the session KPI summary as context and streams the AI response token-by-token via SSE. Provides insights, recommendations, and explanations in plain language.
4	Financial Simulation Engine	Allows What If analysis: users adjust pricing, COGS, or marketing spend via input controls. The system recalculates projected profit, revenue, and margin impact in real time without requiring additional AI API calls.
5	Decision Recommendation Engine	Automatically detects anomalies and inefficiencies in KPI data (e.g., low-margin products, declining revenue trends). Generates structured, actionable recommendations for pricing adjustments, inventory optimization, and cost reduction strategies.

6.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	BizIQ must handle CSV/Excel files up to a few thousand rows without significant performance degradation.	Server-side aggregation via XLSX/PapaParse. Top-N filtering caps AI payload to top/bottom 5 products regardless of dataset size.
Reliability	The AI chat must remain functional during GLM API failures. The dashboard must always render after a successful data upload.	Fallback logic provides rule-based insights (profit trends, underperformers) when GLM is unavailable. Graceful error handling displays user-friendly messages.
Maintainability	Frontend and backend are separated into distinct concerns. The prompt pipeline is modular and independently testable.	Frontend: React components in /src/components/ and /src/pages/. Backend: single server.ts with clear route handlers. Prompt construction isolated from data parsing.

Token Latency	AI responses must begin streaming within 3 seconds under normal conditions at p95 load.	SSE streams tokens as they arrive from GLM API, providing immediate perceived responsiveness. Loading indicator shown while first token is awaited.
Cost Efficiency	Average token usage per user session must not exceed approximately 2,500 tokens.	Only aggregated KPI metrics (not raw rows) sent to GLM. Multi-step prompting avoids redundant context injection. Top-N filtering caps input size. Streaming does not increase token count.
Security	API keys must never be exposed to the client browser at any point.	AI_API_KEY stored in server-side .env file only. Express backend acts as proxy; the frontend never calls the GLM API directly under any circumstances.

6.3. Out of Scope / Future Enhancements

- 6.4.** *Real-time API integrations (e.g., live sales feeds, e-commerce platform sync)*
- 6.5.** *Mobile application (iOS/Android native)*
- 6.6.** *Multi-user accounts and role-based access control*
- 6.7.** *Advanced ML forecasting models (e.g., demand forecasting, time-series regression)*
- 6.8.** *Industry-specific customization and vertical-specific prompt templates*

7. Monitor, Evaluation, Assumptions & Dependencies

7.1. Technical Evaluation:

7.2. Grayscale Rollout & A/B Testing: For UMHackathon 2026, BizIQ is deployed as a single instance. In a post-hackathon production scenario, the recommended strategy is a grayscale deployment where the AI Recommendation Engine is first activated for 10% of user sessions while the remaining 90% receive rule-based fallback insights. After 48-hour monitoring, A/B testing would compare single-step vs. multi-step prompting strategies to determine which produces higher-quality recommendations as rated by SME business owners.

7.3. Emergency Rollback (ER) & Golden Release: The GLM API integration represents the highest-risk component. If the GLM API error rate exceeds 5% over a 10-minute rolling window, an automated rollback would deactivate the AI layer and revert to the rule-based fallback system (Golden Release). The fallback provides profit trend summaries, underperformer flags, and basic margin calculations without any external API calls, ensuring business continuity for SME users during AI service outages.

7.4. Priority Matrix:

Priority	Trigger Condition	Action
P1 Critical	GLM API returns errors on more than 5% of requests within a 10-minute rolling window.	Immediately activate fallback rule-based insight engine. Display user-friendly notification. Alert development team via monitoring system.
P2 High	File parsing fails for more than 10% of uploads (malformed CSV/Excel schema).	Return descriptive validation error to user specifying missing or invalid columns. Log failed file schema for debugging and review.
P3 Medium	AI response latency exceeds 5 seconds at p95 load conditions.	Display loading state with progress indicator. Implement request timeout at 8 seconds. Retry once before showing graceful fallback response.
P4 Low	Dashboard Recharts component fails to render (empty or malformed metric object).	Display empty state with descriptive message. Log render error to console. Chart rerenders automatically on next data refresh.

7.5. Monitoring:

7.6. API Health Monitoring: Log all GLM API calls with response time, token count, and HTTP status code. Alert on sustained error rates above the P1 threshold.

7.7. File Upload Monitoring: Log file size, format, and parse success/failure for every upload. Track average parsing time per file type.

7.8. Token Usage Tracking: Log input and output token counts per session. Flag sessions exceeding 2,500 tokens for prompt optimization review.

- 7.9.** *User Session Metrics: Track average number of AI queries per session, simulation runs per session, and dashboard load time for performance baseline.*
- 7.10.** **Assumptions:**
- 7.11.** *Users have stable internet connections when accessing BizIQ, as GLM API calls require reliable connectivity throughout the session.*
- 7.12.** *Uploaded CSV/Excel files follow a consistent schema with at minimum: product name, revenue, and cost columns. Files lacking these fields will trigger a pre-processing validation error.*
- 7.13.** *The Z AI GLM endpoint remains accessible and within rate limits during the hackathon demo period and subsequent testing phases.*
- 7.14.** *Users upload datasets of moderate size (up to a few thousand rows). Extremely large files exceeding 100,000 rows are outside the supported scope of v1.0.*
- 7.15.** *Business owners interact with the system in English. Multilingual support is not included in BizIQ v1.0.*

7.16. External Dependencies:

Tools	Purpose	Risks
Z AI GLM API (OpenAI SDK)	Core LLM reasoning engine. Handles prompt inference, multi-step reasoning, structured output generation for insights, recommendations, and natural language explanations.	HIGH - GLM API downtime will disable the AI layer entirely. Mitigation: rule-based fallback engine provides basic insights. Retry mechanism re-prompts on incomplete responses. Graceful degradation shown to users.
XLSX Library (npm)	Parses Excel (.xlsx) files server-side into JavaScript row arrays for downstream aggregation and KPI computation.	LOW - Well-maintained open-source library. PapaParse handles CSV if XLSX is unavailable for CSV-formatted data.
PapaParse (npm)	Parses CSV files with automatic delimiter detection and built-in error handling for malformed rows.	LOW - Lightweight, widely used library with broad community support and active maintenance.
Recharts (npm)	React chart library for all KPI dashboard visualizations. Renders from normalized metric objects, not raw AI-generated structures.	LOW - Charts degrade gracefully with empty data states. Frame stability handled at frontend layer via CSS grid layout constraints.
Multer (npm)	Express middleware for handling multipart/form-data file uploads. Stores files temporarily in /uploads/ directory for session processing.	LOW - Files are processed immediately and not retained long-term. Directory is managed and cleaned server-side.
Framer Motion (npm)	Provides UI animations and transitions for the React frontend. Used for page transitions and micro-interactions across all views.	NEGLIGIBLE - Purely cosmetic layer. Its removal would not affect any core business intelligence functionality.

8. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days, structured as a two-phase development sprint.

8.1. Project Timeline:

Period	Milestones & Activities
Day 1-2	Project Planning & Architecture Design. Reviewed PRD requirements, defined system architecture, selected technology stack (React 19, Express, Z AI GLM). Set up monorepo structure (root + biziq directory). Configured development environment and .env variables for GLM API integration.
Day 3-4	Backend Development. Implemented Express server (server.ts) with Multer file upload handling. Built XLSX and PapaParse data parsing pipeline. Designed KPI aggregation engine and Top-N filtering logic. Set up OpenAI SDK integration with Z AI GLM endpoint.
Day 5-6	AI Integration & Prompt Engineering. Developed multi-step prompting pipeline: data summarization -> insight generation -> recommendation formulation. Implemented SSE streaming from backend to frontend. Built fallback rule-based insight engine and graceful error handling.
Day 7-8	Frontend Development. Built all React pages: Dashboard (KPI cards + Recharts), AI Chat (SSE streaming UI), Financial Simulator (What If input controls), Decision Recommendations panel. Applied Tailwind CSS Modern Dark theme and Framer Motion animations.
Day 9	Integration & Testing. End-to-end testing with sample sales_analysis.xlsx dataset. Validated fallback behavior, error handling, and edge cases (malformed files, API timeouts). Performance optimization to reduce average token usage per session.
Day 10	Documentation & Demo Preparation. Completed SAD and PRD documentation following UMHackathon 2026 templates. Prepared live demo with sample business dataset. Final code review and GitHub repository (github.com/nloqmanhn05/BizIQ) cleanup.

8.2. Team Members Role:

Member	Role	Responsibilities
Hakim	Full-Stack Lead & AI Engineer	Designed and implemented server.ts (Express backend), Z AI GLM API integration, SSE streaming pipeline, multi-step prompting strategy, and fallback logic for graceful degradation.
Hakim	Frontend Lead	Built all React pages and UI components (Dashboard, Chat, Simulator, Decision panel). Implemented Recharts visualizations, Tailwind dark theme, and Framer Motion animations.
Vinci Xing Lin	Data Engineer	Developed XLSX and PapaParse parsing pipeline, KPI aggregation engine, Top-N filtering logic, and the data normalization and pre-processing validation layer.

Muhid Farabi , Peter Chan Wei Soon , Melissa Cheong Wei Ling	Product & Documentation Lead	Authored PRD and SAD documentation, managed project timeline, prepared demo dataset (sales_analysis.xlsx), and conducted end-to-end user flow testing.
--	------------------------------	--

8.3. Recommendations:

- 8.4.** *Redis Caching for KPI Summaries: Frequently accessed KPI aggregations (e.g., repeated uploads of the same file) could be cached in Redis for 24 hours to reduce redundant parsing compute and improve dashboard response time.*
- 8.5.** *PostgreSQL Persistent Storage: Future versions should introduce a relational database to persist user session history, uploaded datasets, and AI conversation logs, enabling comparative analytics across multiple time periods.*
- 8.6.** *Prompt Caching: Implement prompt-level caching for repeated query patterns to reduce GLM API token consumption and response latency for common business questions.*
- 8.7.** *WebSocket Upgrade: Replace SSE with WebSocket for bidirectional real-time communication, enabling more interactive AI-driven simulations and multi-user collaborative features.*
- 8.8.** *Containerization: Dockerize the Express backend and React frontend for reproducible deployments across development, staging, and production environments.*